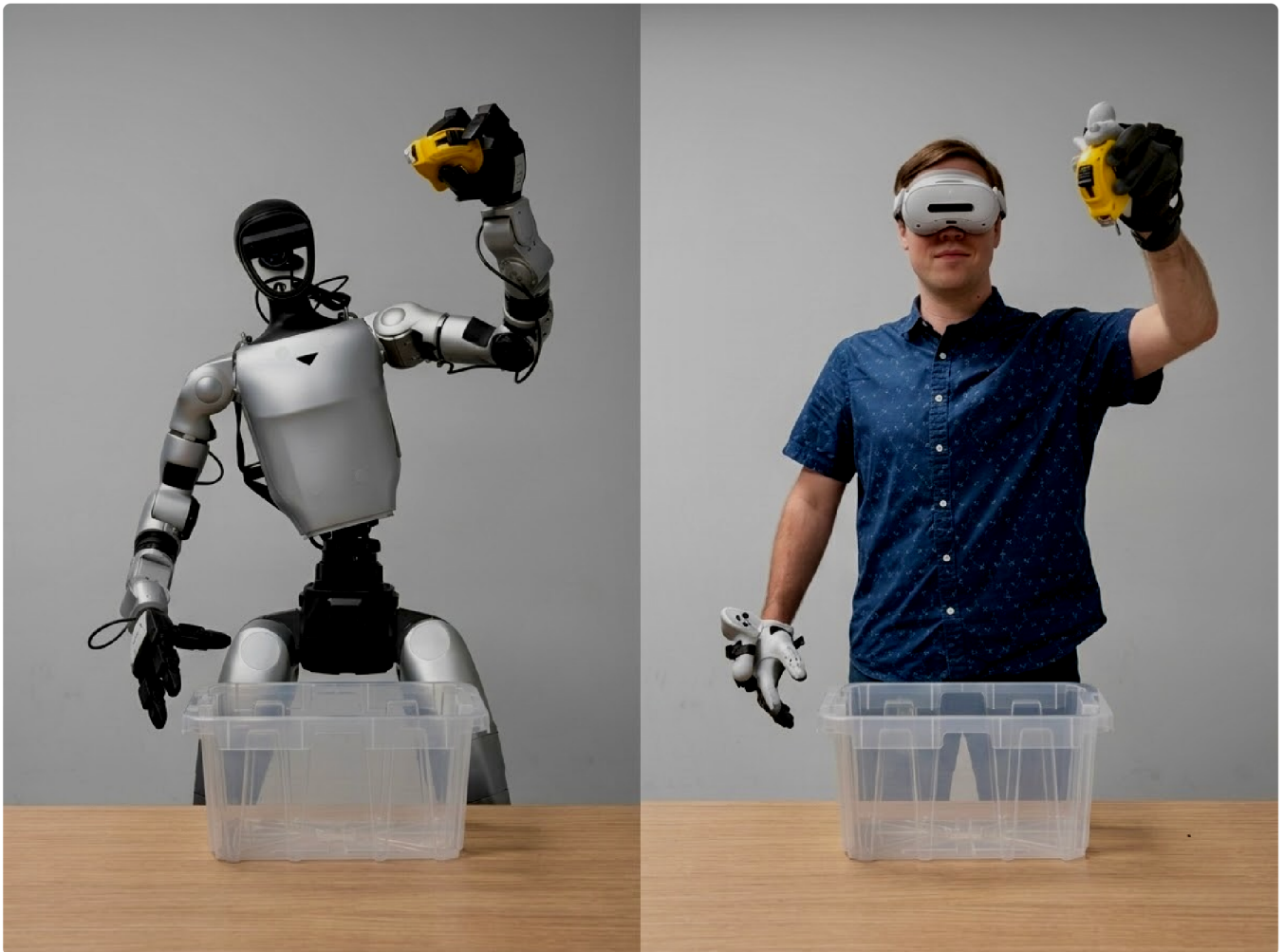


Welcome to Isaac Teleop

License Apache 2.0

Isaac Teleop ([GitHub repo](#)) is the unified framework for high-fidelity egocentric and robot data collection. It provides a standardized device interface, a flexible graph-based retargeting pipeline, and a streamlined data collection workflow that work seamlessly across simulated and real-world robots.



Key features:

- **Unified stack** for sim and real teleoperation (ROS2, Isaac Sim, Isaac Lab)
- **Standardized device interface** for XR headsets, gloves, foot pedals, body trackers
- **Flexible retargeting** to different robot embodiments
- **Plugin system** for adding new devices and use cases
- **Visualization module (Televiz)** for streaming robot camera and sensor feeds, plus 3D reconstruction content to XR headsets
- **Markerless hand reconstruction** from egocentric video into 4D hand and camera poses (monocular supported, stereo on the roadmap)

**Tip**

Just want to get running? Follow the [Quick Start](#) how-to guide for installation and first-run steps.

Table of Contents

Overview

[Isaac Teleop](#)

[Architecture](#)

[Ecosystem](#)

Getting Started

Quick Start

[Try It on Brev — Zero Setup](#)

1. Check out code base (for examples)
2. Install the **isaacteleop** pip package
3. Configure CloudXR (optional)
4. Whitelist ports for Firewall
5. Connect an XR headset
6. Run a teleop example

[Next steps](#)

[More Information](#)

Build from Source

[Prerequisites](#)

[One time setup](#)

1. Clone the repository
2. CMake: Configure and build
3. Running tests
4. Install the **isaacteleop** pip package

Teleop Session

[Overview](#)

[Quick Start](#)



[Configuration Classes](#)

[API Reference](#)

[Examples](#)

[Benefits](#)

[Advanced Features](#)

[Teleop Control State Machine](#)

[Overview](#)

[Default State Machine Behavior](#)

[Connecting Inputs](#)

[Using Context in High-Level App Logic](#)

[Televiz](#)

[Installation](#)

[Overview](#)

[Display modes](#)

[Quick start](#)

[Session configuration](#)

[Layers](#)

[Frame loop](#)

[Session state](#)

[Sharing the XR session](#)

[API reference](#)

[C++ API](#)

[More information](#)

[LeRobot and SO-101](#)

[End-to-end workflow](#)

[In this section](#)

[Pending Tasks](#)

[Contributing](#)

[Previewing documentation changes](#)

[Devices](#)

[Device Trackers](#)

[Data Schema Convention](#)

Tracker Reference

Vendor Selection

Usage Examples

Add a New Device

Step 1: Define the data schema

Step 2: Implement a device plugin

Step 3: Implement a tracker

Step 4: Implement a simple C++ printer (optional)

Schema IO example: build and run

Generic Joint-Space Device

At a glance

Data schema

The SO-101 leader plugin

The reBot DevArm leader plugin

Record and replay

Add another joint-space device

Body Tracking

Setting Up PICO Motion Trackers

How Full Body Tracking Works

Recording and Replay

Quest Body Tracking (Limited Support)

Haptic Feedback

Architecture

Example

Running

Adding a new haptic device

See also

MANUS Gloves

Components

Prerequisites

Installation

Running the Plugin

Data paths

Wrist Positioning — Controllers vs Optical Hand Tracking

Troubleshooting

License

OAK Camera Plugin

Features

Build

Usage

Configuration

Architecture

Dependencies

Output Format

Troubleshooting

OGLO Tactile Glove Plugin

Components

Build

Usage

Recorded data

IMU frame

Tests

Haptikos Exoskeletons

Components

Prerequisites

Quick Start

Important Information

License

References

System Requirements

Teleoperation to Robots with Input Devices

Teleoperation with Isaac Sim and Isaac Lab

Human-centric Data Collection

Build System

Project structure



CMake integration

Using the Python wheel

Output locations

Troubleshooting

Reference

Retargeting Interface

Source Nodes

Available Retargeters

Build a Retargeting Pipeline

Add a New Retargeter

Camera Streaming

Requirements

Setup

First run — no camera required

Supported sources

Running with a real camera

Display modes

Split mode — robot → workstation over RTP

Configuration

Troubleshooting

How it works

Sharing the XR session with TeleopSession

MCAP Recording & Replay

Recording a Live Session

Replaying a Recording

Runnable Example

API Reference

Dedicated CloudXR Runtime

Start the runtime

Optional launch modes

Load CloudXR environment variables

Run teleop examples against the dedicated runtime

Configuration

Rig Launcher

[Prerequisites](#)

[Run a rig](#)

[The rig YAML](#)

[Troubleshooting](#)

Out-of-Band Teleop Control

[Quick start](#)

[ADB automation](#)

[Architecture](#)

[WebSocket protocol](#)

[HTTP API](#)

[Authentication](#)

[Web client integration](#)

[Environment variables](#)

[USB-local mode \(`--setup-oob` `--usb-local`\)](#)

[Troubleshooting](#)

Egocentric Hand Reconstruction

[Video Capture](#)

[Setup](#)

[Hand Reconstruction](#)

[Limitations](#)

License

[Known exceptions](#)

[License headers](#)

Indices and tables

- [Index](#)
- [Search Page](#)

Copyright © 2025-2026, NVIDIA CORPORATION & AFFILIATES, last updated on July 21, 2026.